OPERATING SYSTEM — SESSIONAL 2 CHEAT SHEET

Unit 2: Priority Scheduling → Hyper-Threading + RTOS case study · Unit 3: Concurrency & Deadlock (complete)

Importance: ★★★★★ sure-shot / numerical · ★★★★ very likely long answer · ★★★ likely short answer · ★★ MCQ / one-liner · | **Formulas:** CT = completion time · TAT = CT − AT (total time in system) · WT = TAT − BT (time spent waiting in ready queue) · RT = time first got CPU − AT · Average = sum ÷ n · Need = Max − Allocation

1. PRIORITY SCHEDULING ★★★★★

Definition: A CPU scheduling algorithm in which every process is given a **priority number**, and the CPU is always given to the process with the **highest priority**. If two processes have the same priority, the one that arrived first gets the CPU (FCFS).

Usually smaller number = higher priority (P=1 is more important than P=5). Always read the question – sometimes it says the opposite.

Two types

- **Non-preemptive priority:** once a process gets the CPU it runs till it **finishes**. If a higher-priority process arrives meanwhile, it only goes to the front of the ready queue and waits.
- **Preemptive priority:** if a newly arrived process has **higher priority than the running one, the running process is stopped (preempted)** and put back in the ready queue; the new one runs immediately. → Check priorities at **every arrival**.

Problem: Starvation & its solution Aging

- **Starvation (indefinite blocking):** if high-priority processes keep arriving, a low-priority process may **never get the CPU** – it is ready but waits forever.
- **Aging:** slowly **increase the priority** of a process the longer it waits (e.g. priority improves by 1 every 15 min). Eventually it becomes highest and runs → no starvation.

Advantages: important/system processes run first. **Disadvantages:** starvation; equal priorities need another rule (FCFS).

Numerical (lower number = higher priority)

Process	AT	BT	Priority
P1	0	4	3
P2	1	3	2
P3	2	1	4
P4	3	5	1
P5	4	2	2

(a) Non-preemptive. t=0 only P1 is present → P1 runs fully (0–4). At t=4 ready = P2(2), P3(4), P4(1), P5(2) → **P4** (priority 1). Then P2 & P5 both have priority 2 → P2 (arrived earlier) → P5 → P3.

P1	P4	P2	P5	P3

0 4 9 12 14 15

P	CT	TAT=CT−AT	WT=TAT−BT	RT
P1	4	4	0	0
P2	12	11	8	8
P3	15	13	12	12
P4	9	6	1	1
P5	14	10	8	8
Avg		**44/5 = 8.8**	**29/5 = 5.8**	**5.8**

(b) Preemptive. t=0 P1 starts. t=1 P2(2) arrives, better than P1(3) → **P1 preempted** (rem 3). t=2 P3(4) worse → P2 continues. t=3 P4(1) arrives → **P2 preempted** (rem 1). P4 runs 3–8. Then P2(rem 1) 8–9, P5 9–11, P1(rem 3) 11–14, P3 14–15.

P1	P2	P4	P2	P5	P1	P3

0 1 3 8 9 11 14 15

P	CT	TAT	WT	RT
P1	14	14	10	0
P2	9	8	5	0
P3	15	13	12	12
P4	8	5	0	0
P5	11	7	5	5
Avg		**9.4**	**6.4**	**3.4**

Keywords: priority number, preemption, starvation, indefinite blocking, aging, static vs dynamic priority

2. ROUND ROBIN (RR) ★★★★★

Definition: A **preemptive** scheduling algorithm designed for **time-sharing systems**. Each process gets the CPU for a small fixed time called the **time quantum (time slice, q)**. If it doesn't finish within q, it is preempted and sent to the **back of the ready queue**, and the next process runs. It is basically **FCFS + preemption**.

How to solve RR numericals (step by step)

1. Put the process that arrives first in the ready queue (FIFO).
2. Take the front process, run it for **min(q, remaining burst)**.
3. **Golden rule:** first add all processes that **arrived during** that slice to the queue, **then** add the preempted process at the back.
4. If a process finishes, note its CT. Repeat till queue is empty.

- q **too large** → RR becomes FCFS. q **too small** → too many **context switches** (CPU wastes time switching).
- **Advantages:** fair, **no starvation** (every process gets turn), good response time. **Disadvantages:** lots of context switching; average waiting time often higher than SJF.

Numerical (time quantum q = 2)

Process	P1	P2	P3	P4	P5
AT	0	1	2	3	4
BT	5	3	1	2	3

Time	What happens → ready queue after
0–2	P1 runs (rem 3). P2, P3 arrived → queue: P2, P3, P1
2–4	P2 runs (rem 1). P4, P5 arrived → P3, P1, P4, P5, P2
4–5	P3 runs 1 → **done CT=5** → P1, P4, P5, P2
5–7	P1 runs (rem 1) → P4, P5, P2, P1
7–9	P4 runs 2 → **done CT=9** → P5, P2, P1
9–11	P5 runs (rem 1) → P2, P1, P5
11–14	P2 done 12, P1 done 13, P5 done 14

P1	P2	P3	P1	P4	P5	P2	P1	P5

0 2 4 5 7 9 11 12 13 14

P	CT	TAT	WT	RT
P1	13	13	8	0
P2	12	11	8	1
P3	5	3	2	2
P4	9	6	4	4
P5	14	10	7	5
Avg		**43/5 = 8.6**	**29/5 = 5.8**	**2.4**

Keywords: time quantum / time slice, circular FIFO queue, preemption, context switch, time-sharing, fairness

3. MULTILEVEL QUEUE (MLQ) ★★★

Definition: The ready queue is divided into **several separate queues**, each for one type of process (system, interactive, batch…). A process is **permanently assigned** to one queue when it enters and **never moves** to another queue. Each queue has its **own scheduling algorithm**.

- Example: **foreground** (interactive) queue uses **RR**; **background** (batch) queue uses **FCFS**.
- Scheduling **between** queues: (i) **fixed priority** – serve foreground fully first, then background → background may **starve**; or (ii) **time slice** – e.g. 80% CPU to foreground, 20% to background.
- Typical order (high→low): System → Interactive → Interactive editing → Batch → Student processes.

4. MULTILEVEL FEEDBACK QUEUE (MLFQ) ★★★★

Definition: Like multilevel queue, but processes **can move between queues** based on their behaviour. A process that uses too much CPU is moved **down** to a lower-priority queue; a process that waits too long in a low queue is moved **up** (this is **aging**, prevents starvation).

Defined by: number of queues · algorithm of each queue · when to **upgrade** · when to **demote** · which queue a new process enters.

Example: Q0 = RR (q=8 ms), Q1 = RR (q=16 ms), Q2 = FCFS. A new job enters Q0 and gets 8 ms. If not finished → moved to Q1 and gets 16 more ms. Still not finished → moved to Q2 (FCFS). So short jobs finish fast in Q0; long CPU-bound jobs sink to the bottom.

Advantages: most flexible, low response time, prevents starvation. **Disadvantages:** most **complex**, extra CPU overhead, hard to choose the parameters.

Algorithm	Preemptive?	Starvation?	Selects by
FCFS	No	No (convoy effect)	Arrival time
SJF	No	Yes	Burst time
SRTF	Yes	Yes	Remaining BT
Priority	Both	Yes → aging	Priority no.
Round Robin	Yes	No	Time quantum
MLQ	Both	Yes	Fixed queue
MLFQ	Yes	No (aging)	Feedback

5. PROCESS vs THREAD ★★★★

Process: a program in execution, with its **own** memory (code, data, heap, stack), files and PCB.
Thread: the **basic unit of CPU utilization** – a single path of execution **inside** a process. Called a **lightweight process**. Each thread has its **own** thread ID, program counter, register set and stack, but **shares** the code, data, heap and open files of its process with other threads.

Process	Thread
Heavyweight; own address space	Lightweight; shares process's address space
Creation & context switch are slow/costly	Creation & switch are fast/cheap
Needs IPC to talk to other processes	Talks through shared memory directly
Independent – one crashing doesn't affect others	One faulty thread can crash whole process
Has its own PCB	Has a TCB inside the process

Thread States ★★

- **New** – thread created but not yet started (`new Thread()`).
- **Runnable / Ready** – ready to run, waiting for CPU.
- **Running** – currently executing on CPU.
- **Blocked / Waiting / Timed-waiting** – waiting for I/O, a lock, sleep() or join(); returns to Runnable when event occurs.
- **Terminated / Dead** – finished or crashed; **cannot be restarted**.

Benefits of Multithreading ★★★ (remember "RRES")

- **Responsiveness** – if one thread blocks (e.g. downloading), others keep the app responsive (UI doesn't freeze).
- **Resource sharing** – threads share memory & resources of the process automatically; no IPC needed.
- **Economy** – creating/switching threads is much cheaper than processes.
- **Scalability** – threads can run in parallel on multiple cores.

6. TYPES OF THREADS ★★★

User-Level Thread (ULT): created and managed by a **thread library in user space**; the kernel doesn't even know they exist.
Kernel-Level Thread (KLT): created and managed **directly by the OS kernel**; kernel schedules each thread.

User-Level Thread	Kernel-Level Thread
Fast to create/switch, no system call	Slower (needs kernel involvement)
If one thread blocks, whole process blocks	If one blocks, others continue
Can't use multiple CPUs in parallel	Can run on different CPUs
Ex: POSIX Pthreads, Mach C-threads, Solaris threads	Ex: Windows NT/2000, Linux, Solaris, Tru64

7. MULTITHREADING MODELS ★★★★

Describe **how user threads are mapped to kernel threads**.

- **Many-to-One:** many user threads → **one** kernel thread. Thread management is fast, but if one thread makes a blocking call **all block**, and threads **can't run in parallel** on multicore. Ex: Solaris Green Threads, GNU Portable Threads.
- **One-to-One: each** user thread → its **own** kernel thread. More concurrency, true parallelism, one blocking doesn't stop others. Drawback: creating a user thread needs a kernel thread (overhead), so number of threads is limited. Ex: Windows, Linux, OS/2.
- **Many-to-Many:** many user threads → a **smaller or equal number** of kernel threads (mapped dynamically). Best of both: create as many threads as needed, they run in parallel, a blocking call doesn't block all. Ex: older Solaris, Windows ThreadFiber.

8. HYPER-THREADING (HT) ★★★

Definition: Intel's technology (a form of **Simultaneous Multithreading – SMT**) that makes **one physical CPU core appear as two logical processors** to the OS, so the core can execute instructions of **two threads at the same time**.

- **Why?** A core has many execution units (integer, floating-point, load/store). One thread alone leaves many of them idle (waiting for memory, cache miss).
- **How?** Each logical processor has its own **architectural state (registers)**, but both **share** execution units and cache. When thread 1 stalls, thread 2 uses the idle units.
- **Benefits:** higher throughput, better utilization, parallelism without adding extra cores (cheap). Gives ~15–30% gain, **not 2×**.

9. CASE STUDY: CPU SCHEDULING IN RTOS ★★

Real-Time Operating System (RTOS): an OS where a task's result must be produced **within a strict deadline**; correctness depends on **time** as well as result. Key traits: **determinism, predictability, low latency**. Ex: FreeRTOS, VxWorks, QNX, RT-Linux, Zephyr.

- **Hard real-time:** missing a deadline = system failure (airbag, ABS, pacemaker). **Soft real-time:** missing a deadline only reduces quality (video streaming).
- **Rate Monotonic (RMS):** static priority; task with **shorter period** gets higher priority. Schedulable if $U = \Sigma(Ci/Pi) \le n(2^{1/n} - 1)$ (n=2 → 0.828).
- **Earliest Deadline First (EDF):** dynamic priority; task whose **deadline is nearest** runs first. Optimal; schedulable if $U \le 1$.
- **Deadline Monotonic (DMS):** static; **shorter relative deadline** = higher priority.
- **Priority inversion:** a high-priority task waits because a low-priority task holds a resource it needs. Fix: **Priority Inheritance** (low task temporarily gets the high priority) or **Priority Ceiling**.
- Terms: **WCET** (worst-case execution time), **jitter** (variation in timing), **tick rate**.

10. CONCURRENCY & RACE CONDITION ★★★★

Concurrency: running multiple processes/threads in **overlapping time periods**. They need not run at the exact same instant – the OS can **interleave** them on one CPU. Types: process-level, thread-level, distributed.

Race Condition: a situation where two or more processes access and change **shared data at the same time**, and the **final result depends on the order** in which they happen to execute. Different timings give different (wrong) answers.

Example: counter = 5. P1 does `counter++`, P2 does `counter--`. In machine code each is 3 steps: `reg = counter; reg = reg ± 1; counter = reg`. If they interleave (P1 reads 5, P2 reads 5, P1 writes 6, P2 writes 4), the final value is **4 or 6 instead of 5**.

Effects: data inconsistency, unpredictable, hard-to-debug results.
Prevention: allow only one process at a time in the shared code → locks, mutex, semaphores, atomic instructions.

11. CRITICAL SECTION PROBLEM ★★★★★

Critical Section (CS): the part of a program where a process **accesses shared resources** (shared variables, files, tables). The **critical section problem** is to design a protocol so that **no two processes are in their critical sections at the same time**.

```
do {
   entry section      // ask permission to enter
      critical section
   exit section       // announce leaving
      remainder section
} while (true);
```

Any correct solution must satisfy 3 conditions:
1. Mutual Exclusion – if one process is executing in its CS, **no other** process can be in its CS.
2. Progress – if no process is in the CS and some want to enter, the choice of who enters next cannot be postponed forever, and only processes that want to enter take part in the decision (a process in its remainder section cannot block others).
3. Bounded Waiting – after a process requests entry, there is a **limit** on how many times other processes can enter before it. (→ no starvation)

12. INTER-PROCESS COMMUNICATION (IPC) ★★★

IPC: mechanism that lets **cooperating processes** (processes that affect each other) **exchange data and synchronize**. Needed for information sharing, speed-up, modularity, convenience. Two models:

Shared Memory	Message Passing
A common memory region is created; processes read/write it	Processes exchange messages using `send()` / `receive()`
Faster – kernel needed only to set it up	Slower – every message is a system call
Good for large data	Good for small data & distributed systems
Programmer must handle synchronization	Easier; OS handles it

- **Direct communication:** name the other process explicitly – `send(P, msg)`, `receive(Q, msg)`; one link between each pair.
- **Indirect communication:** messages go through a **mailbox (port)** – `send(A, msg)`, `receive(A, msg)`; many processes can share a mailbox.

13. LOCK VARIABLE ★★★★★

Definition: the simplest **software** synchronization method. A shared integer `lock` is used: **0 = CS is free, 1 = CS is busy**. A process waits while lock is 1, sets it to 1 when it enters, and resets to 0 when it leaves. Works in user mode for any number of processes.

```
int lock = 0;          // shared by all
while (1) {
   while (lock == 1);  // entry: busy wait
   lock = 1;           // take the lock
   /* Critical Section */
   lock = 0;           // exit: release
   /* Remainder Section */
}
```

× It does NOT guarantee Mutual Exclusion!
Why it fails: P1 checks lock → finds 0 → **gets preempted before writing lock = 1**. P2 now checks lock → also 0 → sets 1 → enters CS.

P1 resumes, sets lock = 1, and also enters CS → **both in CS**. Reason: **checking and setting the lock are two separate steps, not atomic**.
Other drawbacks: **busy waiting** (waiting process keeps looping and wastes CPU), no bounded waiting. ✓ Progress is satisfied. **Fix:** make check+set atomic → Test-and-Set instruction, mutex lock, semaphore.

Mutex Lock

A lock with two **atomic** operations: `acquire(){ while(!available); available = false; }` and `release(){ available = true; }`. A mutex that busy-waits is called a **spinlock**. Example: only one ATM can update a bank balance at a time.

Strict Alternation (turn variable) ★★★

Shared `int turn = 0;` – processes take turns strictly: P0, P1, P0, P1…

```
P0: while(turn != 0); /*CS*/ turn = 1;
P1: while(turn != 1); /*CS*/ turn = 0;
```

✓ Mutual exclusion. × **Progress fails**: if it's P1's turn but P1 doesn't want to enter, P0 can't enter again even though CS is empty. Only 2 processes; busy waiting.

14. PETERSON'S SOLUTION ★★★★

Definition: a classic **software solution for 2 processes** that satisfies all 3 CS conditions. Uses two shared variables: `flag[2]` – flag[i]=true means "Pi **wants** to enter"; and `turn` – whose **turn** it is if both want.

```
// process Pi (other process is j)
do {
   flag[i] = true;   // I want to enter
   turn = j;         // let other go first
   while (flag[j] && turn == j);
         // wait while other wants & it's his turn
   /* Critical Section */
   flag[i] = false;  // I'm done
   /* Remainder Section */
} while (true);
```

Why it works: if both want to enter, `turn` can hold only one value, so only one passes the while loop. ✓ Mutual Exclusion ✓ Progress ✓ Bounded Waiting.
Drawbacks: only 2 processes, busy waiting, may fail on modern CPUs that reorder instructions.

Bakery Algorithm (Lamport) ★★

Solution for **n processes**, like taking a token at a bakery: each process wanting CS takes a number `number[i] = max(number[]) + 1`; the **smallest number enters first**; if two have the same number, the **smaller process ID** goes first. Uses `choosing[]` and `number[]`. FCFS order, no special hardware, but busy waiting.

Test-and-Set (TSL) – Hardware solution ★★★

A special CPU instruction that **reads the old value of lock and sets it to true in one indivisible (atomic) step** – so no other process can sneak in between (fixes the lock-variable problem).

```
bool TestAndSet(bool *target) { // ATOMIC
   bool rv = *target;
   *target = true;  return rv; }
do { while (TestAndSet(&lock)); // spin
     /* CS */   lock = false;
     /* remainder */ } while(1);
```

✓ Mutual exclusion. × busy waiting (spinlock), × bounded waiting not guaranteed.

15. SEMAPHORES ★★★★

Definition (Dijkstra): a semaphore **S** is an **integer variable** that, apart from initialization, is accessed only through two **atomic** operations: **wait()** (also P / down) and **signal()** (also V / up).

```
wait(S)   { while (S <= 0);  S--; } // take
signal(S) { S++; }                  // release
```

- **Binary semaphore:** value only 0 or 1 – works like a mutex lock (mutual exclusion).
- **Counting semaphore:** any value – used to control a resource that has **n instances** (initialize S = n).
- **Ordering use:** to make S1 (in P1) run before S2 (in P2): `synch = 0`; P1: S1; signal(synch); P2: wait(synch); S2.
- **Without busy waiting:** wait() puts the process in the semaphore's waiting queue (**block()**); signal() wakes one up (**wakeup()**). A negative value = number of processes waiting.
- **Wrong use causes deadlock/starvation:** writing signal before wait, wait twice, or forgetting wait/signal.

16. BOUNDED BUFFER (PRODUCER–CONSUMER) ★★★★★

Problem: a **producer** process creates items and puts them into a shared buffer of fixed size **n**; a **consumer** process takes items out. Rules: (1) producer must **wait if the buffer is full**, (2) consumer must **wait if the buffer is empty**, (3) both must **not access the buffer at the same time**.

Semaphore	Initial	Meaning
mutex	1	only one process touches buffer at a time
empty	**n**	number of empty slots (all empty at start)
full	**0**	number of filled slots (none at start)

```
PRODUCER                        CONSUMER
do {                            do {
 produce item;                   wait(full);   // item?
 wait(empty); // free slot?      wait(mutex);  // lock
 wait(mutex); // lock            remove item;
 add item to buffer;             signal(mutex);// unlock
 signal(mutex);// unlock         signal(empty);// +1 slot
 signal(full); // +1 item        consume item;
} while (true);                 } while (true);
```

▲ **Order matters: if producer does wait(mutex) before wait(empty) when buffer is full, it holds the lock while sleeping → consumer can never enter → DEADLOCK.**

Unbounded buffer = no size limit (producer never waits). Using a plain `counter++ / counter--` instead of semaphores causes a race condition.

17. READERS–WRITERS PROBLEM ★★★★★

Problem: a shared data set (e.g. a database) is used by **readers** (only read) and **writers** (read & modify). **Many readers can read at the same time** (reading doesn't change data), but a **writer needs exclusive access** – no other writer or reader may be present while it writes.

Shared: `semaphore rw_mutex = 1;` (writer's lock) `semaphore mutex = 1;` (protects read_count) `int read_count = 0;` (readers currently reading)

```
WRITER                  READER
do {                    do {
  wait(rw_mutex);         wait(mutex);
  /* writing */           read_count++;
  signal(rw_mutex);       if (read_count == 1)
} while (true);              wait(rw_mutex); // 1st reader
                        signal(mutex);     // blocks writer
                        /* reading */
                        wait(mutex);
                        read_count--;
                        if (read_count == 0)
                           signal(rw_mutex); // last reader
                        signal(mutex);    // lets writer in
                      } while (true);
```

- **Logic:** the **first reader locks out writers**; later readers just enter; the **last reader to leave lets writers in**.
- **First R-W problem (Readers' preference)** – the code above. No reader waits unless a writer is already writing → **writers may starve**.
- **Second R-W problem (Writers' preference)** – once a writer is waiting, **no new reader** may start → **readers may starve**.

18. DINING PHILOSOPHERS PROBLEM ★★★★★

Problem: 5 philosophers sit around a round table with a bowl of rice and **5 chopsticks** (one between each pair). A philosopher either **thinks** or **eats**. To eat, he needs **both** the left and right chopstick, picking them up one at a time; after eating he puts both down. It shows how to allocate several resources among processes **without deadlock and starvation**.

```
semaphore chopstick[5] = {1,1,1,1,1};
// Philosopher i
do {
   wait(chopstick[i]);            // pick left
   wait(chopstick[(i+1) % 5]);    // pick right
   /* eat */
   signal(chopstick[i]);          // put left
   signal(chopstick[(i+1) % 5]);  // put right
   /* think */
} while (true);
```

Problem: if all 5 get hungry together and each picks up his left chopstick, every right chopstick is taken → everyone waits forever → DEADLOCK (circular wait). Starvation also possible.
Solutions: (1) allow at most **4** philosophers at the table at once; (2) pick up chopsticks only if **both are available** (do the picking inside a critical section); (3) **asymmetric**: odd philosopher picks left first then right, even picks right first then left; (4) monitor solution with states THINKING / HUNGRY / EATING.

Sleeping Barber Problem ★★

A shop has 1 barber, 1 barber chair and **N waiting chairs**. No customers → barber **sleeps**. A customer arriving wakes the barber; if barber is busy he sits in a free chair; if all chairs are full he **leaves**. Semaphores: `customers = 0` (waiting customers), `barber = 0` (barber ready), `mutex = 1`; int `waiting = 0`.

```
Barber:  while(1){ wait(customers); wait(mutex);
           waiting--; signal(barber);
           signal(mutex); cut_hair(); }
Customer: wait(mutex);
  if (waiting < N) { waiting++; signal(customers);
     signal(mutex); wait(barber); get_cut(); }
  else signal(mutex);      // shop full, leave
```

19. DEADLOCK ★★★★★

Definition: a situation where a set of processes are **blocked forever** because **each process is holding a resource and waiting for another resource held by some other process** in the same set. None can proceed. A process uses a resource as: **request → use → release**.

Example: P1 holds the printer and wants the scanner; P2 holds the scanner and wants the printer → both wait forever. (DB: T1 locks table A wants B, T2 locks B wants A.)

4 Necessary Conditions (Coffman) – deadlock happens only if ALL 4 hold at once:
1. Mutual Exclusion – at least one resource is non-sharable (only one process can use it at a time).
2. Hold and Wait – a process holds at least one resource while waiting for others.
3. No Preemption – a resource can't be taken away; it is released only voluntarily by the holder.
4. Circular Wait – a circular chain P0 waits for P1, P1 for P2, …, Pn waits for P0.

Resource Allocation Graph (RAG)

A graph showing which process holds/requests which resource. Circle = process, rectangle = resource type (dots = instances). **Request edge** Pi → Rj (Pi wants Rj); **Assignment edge** Rj → Pi (Pi holds Rj); **Claim edge** Pi ⇢ Rj dashed (may request in future).

- No cycle ⇒ no deadlock.
- Cycle + **one instance** per resource type ⇒ **deadlock certain**.
- Cycle + **several instances** ⇒ deadlock **possible**, not certain.

Methods for handling deadlock

(1) **Prevention** – ensure one of the 4 conditions can never hold. (2) **Avoidance** – check every request and never enter an unsafe state. (3) **Detection & Recovery** – allow deadlock, detect it, then recover. (4) **Ignore** it (Ostrich algorithm – used by Windows/UNIX, since deadlocks are rare).

Deadlock Prevention ★★★★ – break one condition

- **Mutual Exclusion:** make resources sharable (e.g. read-only files). Not possible for printers, so usually can't be broken.
- **Hold & Wait:** process must request **all** its resources before starting, or request only when it holds **none**. Drawback: low resource utilization, starvation.
- **No Preemption:** if a process holding resources requests one that isn't free, it must **release all** it holds and restart later.
- **Circular Wait:** number all resource types and force processes to request in **increasing order** only → no cycle can form. (Most practical.)

Deadlock Avoidance

Each process declares in advance the **maximum** resources it may need. Before granting a request, the OS checks that the system stays in a **safe state**.

Safe state: there exists at least one order (**safe sequence**) <P1, P2, …, Pn> in which every process can get its remaining need from available resources + those released by earlier processes, and finish. Safe ⇒ no deadlock. Unsafe ⇒ deadlock possible (not certain).

Single instance of each resource → **RAG with claim edges** (grant request only if converting it to an assignment edge makes no cycle). Multiple instances → **Banker's algorithm**.

Deadlock Detection ★★★

- **Single instance:** build a **wait-for graph** (remove resource nodes; edge Pi → Pj means Pi waits for Pj). If it has a **cycle → deadlock**. Cycle detection costs O(n²).
- **Multiple instances:** algorithm like Banker's safety check using a **Request** matrix; any process that can't finish is deadlocked.

Recovery from Deadlock

- **Process termination:** abort **all** deadlocked processes, or abort **one at a time** until the cycle breaks (choose by priority, how long it ran, resources held/needed, interactive or batch).
- **Resource preemption:** take resources away – **select a victim** (minimum cost), **rollback** it to a safe state and restart, and avoid **starvation** (same process shouldn't always be the victim – count rollbacks).

Case study: Oracle / MySQL InnoDB – T1 locks row 101 and waits for row 202 while T2 locks 202 and waits for 101 → circular wait. DB detects it via wait-for graph and rolls back one transaction (victim) with a deadlock error.

20. BANKER'S ALGORITHM ★★★★★

Definition: a **deadlock-avoidance** algorithm for resources with **multiple instances**. Like a bank that never lends money in a way that it can't satisfy all customers. Each process declares its **maximum need**; a request is granted **only if the system stays in a safe state** after granting it.

Data structures (n processes, m resource types): **Available[m]** – free instances of each type; **Max[n][m]** – max demand of each process; **Allocation[n][m]** – currently given; **Need[n][m]** = Max − Allocation (still may ask). Also **Total = Available + sum of Allocation**.

Safety Algorithm (is the state safe?)

1. Work = Available; Finish[i] = false for all i.
2. Find a process i with Finish[i] = false and **$Need_i \le Work$**. If none, go to step 4.
3. Pretend it finishes and returns its resources: **Work = Work + $Allocation_i$**; Finish[i] = true; go to step 2.
4. If all Finish[i] = true → **SAFE** (the order you picked is the safe sequence); otherwise **UNSAFE**.

Resource-Request Algorithm (Pi asks $Request_i$)

1. If $Request_i \le Need_i$ go on; else **error** (asked more than its max).
2. If $Request_i \le$ Available go on; else Pi must **wait**.
3. **Pretend to allocate:** Available −= Request; $Allocation_i$ += Request; $Need_i$ −= Request. Run safety algorithm: safe → **grant**; unsafe → undo and Pi **waits**.

Numerical 1 — 5 processes, A=10, B=5, C=7

P	Allocation A B C	Max A B C	Need = Max − Alloc
P0	0 1 0	7 5 3	7 4 3
P1	2 0 0	3 2 2	1 2 2
P2	3 0 2	9 0 2	6 0 0
P3	2 1 1	2 2 2	0 1 1
P4	0 0 2	4 3 3	4 3 1

Available = Total − ΣAllocation = (10,5,7) − (7,2,5) = **(3,3,2)**

Step	Process	Need ≤ Work ?	Work = Work + Alloc
1	P1	(1,2,2) ≤ (3,3,2) ✓	(5,3,2)
2	P3	(0,1,1) ≤ (5,3,2) ✓	(7,4,3)
3	P0	(7,4,3) ≤ (7,4,3) ✓	(7,5,3)
4	P2	(6,0,0) ≤ (7,5,3) ✓	(10,5,5)
5	P4	(4,3,1) ≤ (10,5,5) ✓	(10,5,7)

✅ All finish → **SAFE**, safe sequence **<P1, P3, P0, P2, P4>**. (Other valid orders like <P1,P3,P4,P2,P0> are also correct.)
Can P1's request (1,0,2) be granted? Step 1: (1,0,2) ≤ Need P1 (1,2,2) ✓. Step 2: (1,0,2) ≤ Available (3,3,2) ✓. Step 3: pretend → Available = (2,3,0), Allocation P1 = (3,0,2), Need P1 = (0,2,0). Safety check gives <P1, P3, P4, P0, P2> → safe → **GRANT**. (After this, P4 asking (3,3,0) must **wait** – more than Available; P0 asking (0,2,0) is **denied** – resulting state unsafe.)

Numerical 2 — 4 processes (X Y Z), Available = (1,1,2)

P	Allocation	Max	Need
P0	2 1 1	5 1 3	3 0 2
P1	1 0 1	2 1 2	1 1 1
P2	4 1 2	7 2 3	3 1 1
P3	1 0 0	2 1 1	1 1 1

Work (1,1,2) → **P1** (1,1,1 ≤ 1,1,2) → Work (2,1,3) → **P3** (1,1,1 ≤ 2,1,3) → (3,1,3) → **P0** (3,0,2 ≤ 3,1,3) → (5,2,4) → **P2** (3,1,1 ≤ 5,2,4) → (9,3,6).
✅ **SAFE**, safe sequence **<P1, P3, P0, P2>**. Check: final Work (9,3,6) = Total resources.

Keywords: deadlock avoidance, a-priori maximum claim, safe sequence, Need ≤ Work, Work += Allocation, pretend allocation

⚡ EXAM QUICK-HITS (MCQ) ★★★★

- Module that gives CPU to the process chosen by short-term scheduler = **dispatcher**.
- Semaphore is an **integer variable**; two kinds: **binary & counting**.
- Bounded buffer problem is also called **Producer–Consumer**.
- Circular wait is prevented by **linear ordering of resource types**.
- Dining philosophers occurs with **5 philosophers and 5 chopsticks**.
- In readers–writers, **writers** are given exclusive access.
- Edge Pi → Pj in wait-for graph means **Pi is waiting for Pj**; used when each resource has a **single instance**.
- No cycle in RAG ⇒ system is **not deadlocked**. Deadlock-avoidance algorithm = **Banker's**.
- Round Robin is a scheduling algorithm – it does **not** guarantee mutual exclusion.
- Thread shares **memory space** with its process; HT gives **2 logical processors** per core and duplicates **registers**.
- RTOS must provide **predictable timing**. MLFQ uses **dynamic feedback**.
- Busy waiting = **spinlock**: wastes CPU but avoids context switch (fine for very short waits on multicore).